ІНЖЕНЕРНА СПЕЦИФІКАЦІЯ · ВНУТРІШНІЙ ДОКУМЕНТ

# LoadBoardGram — технічна документація

Архітектура системи, доменна модель, AI-підсистема та інженерні рішення — від того, що вже працює, до того, що спроєктовано на масштаб.

Версія документа **0.1** Дата **17.08.2026** Статус продукту **MVP / демо**

Пов'язаний документ **Project Overview (для інвесторів)**

## 01 Про документ

Це технічна специфікація проєкту LoadBoardGram — вантажної біржі з прямим з'єднанням вантажовідправника і перевізника, без брокера-посередника. На відміну від інвесторського огляду (`project-documentation.html`), тут немає маркетингового спрощення: документ описує систему так, як її бачить інженерна команда — включно з тим, що ще не побудовано.

Аудиторія: технічні співзасновники, майбутні інженери, що приєднуються до команди, і технічний due diligence з боку інвестора.

Кожен розділ і кожен рядок таблиці позначені міткою зрілості:

MVP · ГОТОВО — реалізовано і працює в поточному демо-каркасі· ФАЗА 2 — у найближчому спринті розробки· ФАЗА 3 — заплановано, дизайн ще не заморожений· R&D — потребує дослідження/прототипування моделі перед комітом у продукт· РИЗИК — відомий технічний борг або відкрите питання.

Документ навмисно не приховує незрілість поточної кодової бази: MVP — це навмисно спрощений транзакційний каркас (реєстрація → публікація вантажу → взяття вантажу → завершення), без жодної AI-функції, зібраний, щоб перевірити базовий цикл shipper↔carrier до інвестиції в ML-шар.

## 02 Огляд системи та рівень зрілості

LoadBoardGram — двостороння платформа: вантажовідправник публікує вантаж, перевізник бачить його і бере напряму, без брокера. Довгострокове бачення — не лише дошка оголошень, а система, де AI виконує роботу диспетчера й брокера: підбирає вантаж, прогнозує ставку, веде перші переговори, перевіряє контрагента і обробляє документи.

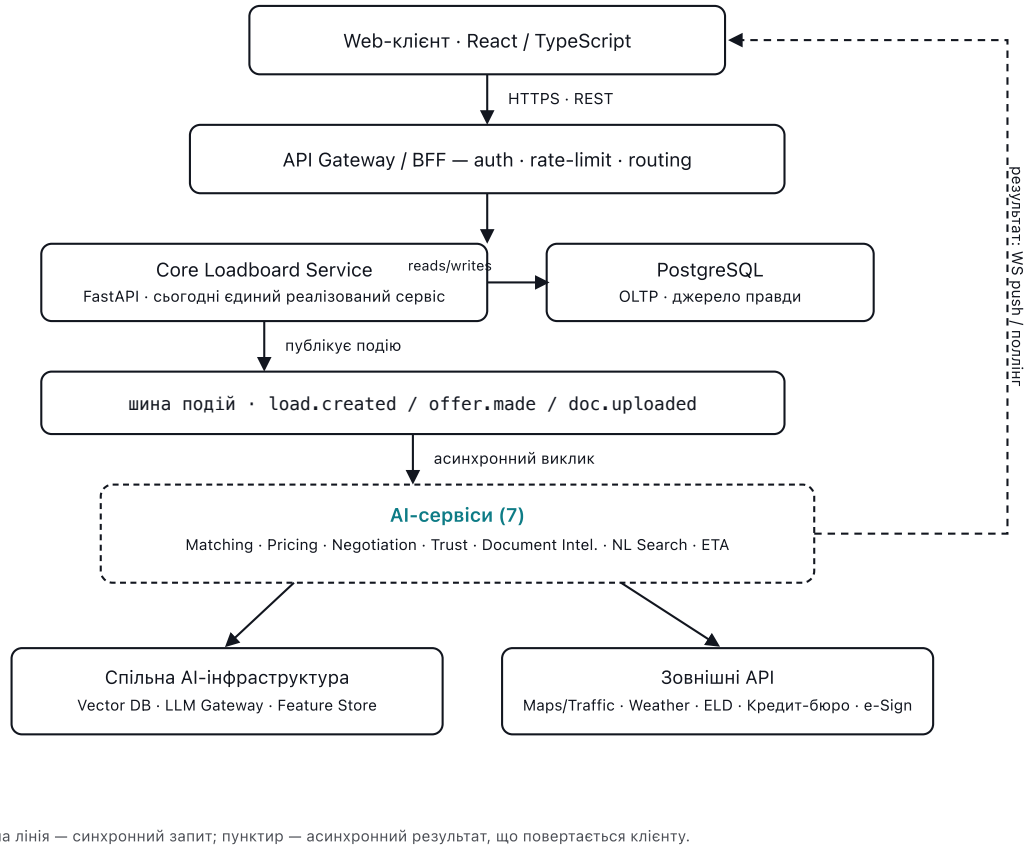
| КОМПОНЕНТ | ПОТОЧНИЙ СТАН (MVP) | ЦІЛЬОВИЙ СТАН |
|---|---|---|
| Клієнт | React SPA з клієнтським роутингом (`react-router`): маркетингова головна, список вантажів, сторінка деталей вантажу, сторінка документації, ручний `fetch` | React SPA + мобільний застосунок (React Native), кешування запитів, realtime через WebSocket |
| Бекенд | Один монолітний FastAPI-сервіс | Core-сервіс + вісім AI-сервісів, розв'язаних через шину подій |
| База даних | Один PostgreSQL, 3 таблиці (`users`, `loads`, `refresh_tokens`), схема під Alembic-міграціями | PostgreSQL (OLTP) + pgvector/векторна БД + Redis (кеш/черги) + об'єктне сховище документів |
| Автентифікація | Email + пароль, короткоживучий JWT (15 хв) + revocable refresh-токен з ротацією (розділ 9) | Refresh-токен в httpOnly cookie замість `localStorage`, MFA, SSO для enterprise-акаунтів |
| Матчинг вантажів | MVP: детерміністична евристика під історію перевізника (розділ 7.3) | ML-ранжування під профіль перевізника |
| AI-функції | 3 з 7 — NL Search, Smart Matching і ETA, усі MVP без ML/embeddings (розділи 7.3/7.6/7.7) | 7 AI-сервісів (розділ 7) |
| Інфраструктура | Docker Compose, один хост | Kubernetes, кілька середовищ, автоскейлінг |
| Спостережуваність | Немає — лише логи `uvicorn` у stdout | Метрики, трейсинг, структуровані логи, алертинг, AI-eval пайплайни |

> **Навмисне рішення, не недогляд.** AI не додається «зверху» на готову дошку оголошень — уся архітектура нижче спроєктована так, щоб AI-сервіси підключалися як окремі, незалежно деплойовані споживачі подій, а не як виклики всередині монолітного контролера. Це головна архітектурна ставка проєкту.

## 03 Архітектура верхнього рівня

Система розділена на дві половини з різними вимогами до надійності й латентності. **Core Loadboard Service** — транзакційне ядро (реєстрація, вантажі, статуси, платежі в майбутньому): має

бути швидким, консистентним, з жорсткими SLA. **AI-сервіси** — обчислювально важкі, залежать від зовнішніх LLM/ML-провайдерів, можуть відповідати з затримкою в секунди і не повинні блокувати транзакційний шлях. Тому вони зв'язані асинхронно через шину подій, а не прямими синхронними викликами.




Мал. 1 — Транзакційне ядро і AI-сервіси розв'язані шиною подій: Core пише у Postgres і публікує подію, AI-сервіси споживають її асинхронно й повертають результат клієнту окремим каналом (WS/поллінг), не блокуючи основний запит.

Наразі реалізований лише прямокутник **Core Loadboard Service ↔ PostgreSQL** у верхній половині схеми — без API Gateway, без шини подій, без жодного AI-сервіса. Решта — цільова архітектура, під яку вже зараз проєктується доменна модель (розділ 5) і контракти подій.

## Чому шина подій, а не прямі виклики

- **Ізоляція збоїв.** Якщо LLM-провайдер деградує чи впав, публікація вантажу і взяття вантажу продовжують працювати — це різні відмовостійкі домени.
- **Незалежне масштабування й деплой.** Negotiation Agent можна викотити окремо від Core, дати йому свій пул воркерів і GPU/API-бюджет, не чіпаючи транзакційний код.
- **Заміна моделі без даунтайму ядра.** Провайдер LLM або алгоритм скорингу можна змінити, підключивши нову версію споживача до тієї самої події.

- **Аудит за замовчуванням.** Кожна подія — це запис у журналі: хто що запропонував і коли, важливо для спорів «AI щось пообіцяв контрагенту».

## 04 Технологічний стек

### Клієнт

| ТЕХНОЛОГІЯ | РОЛЬ | СТАТУС |
|---|---|---|
| React 18 + TypeScript | SPA, компонентний UI | MVP |
| Vite | Dev-сервер і білд | MVP |
| Ручний `fetch` + React Context | Доступ до API та стан автентифікації (`AuthContext.tsx`) | MVP |
| react-router-dom | Клієнтський роутинг — список вантажів (`/`) і сторінка деталей (`/loads/:id`) | MVP |
| Leaflet + CARTO Voyager tiles | Мапа на сторінці деталей вантажу — маркери origin/destination і приблизний маршрут | MVP |
| OpenStreetMap Nominatim | Геокодування origin/destination у координати — безкоштовно, без ключа (`frontend/src/geocode.ts`) | MVP |
| OSRM (публічний demo-сервер) | Реальний маршрут по дорогах, дистанція й час у дорозі — безкоштовно, без ключа (`frontend/src/routing.ts`); фолбек на пряму лінію, якщо недоступний | MVP |
| TanStack Query | Кешування запитів, повторні спроби, інвалідація | ФАЗА 2 |
| WebSocket-клієнт | Realtime-оновлення статусів вантажу, офери AI-агента | ФАЗА 2 |
| React Native | Мобільний застосунок для перевізників (сповіщення, GPS) | ФАЗА 3 |

## Бекенд і дані

| ТЕХНОЛОГІЯ | РОЛЬ | СТАТУС |
|---|---|---|
| Python 3.12 + FastAPI | REST API, валідація через Pydantic | MVP |
| SQLAlchemy 2.x | ORM, декларативні моделі | MVP |
| PostgreSQL 16 | Основне сховище (OLTP) | MVP |
| python-jose + passlib/bcrypt | JWT, хешування паролів | MVP |
| Redis | Кеш сесій, rate-limit лічильники, черга задач (Celery/RQ broker) | ФАЗА 2 |
| pgvector (розширення Postgres) | Векторний пошук для NL-запитів та матчингу — старт без окремої векторної БД | ФАЗА 2 |
| Kafka / Redis Streams | Шина подій між Core і AI-сервісами | ФАЗА 3 |
| S3-сумісне об’єктне сховище | Зберігання документів (BOL/POD/rate confirmation) | ФАЗА 2 |
| Alembic | Міграції схеми БД — застосовуються перед стартом застосунку (`backend/entrypoint.sh` у Docker, CI-крок, автотест-фікстура), не самим застосунком | MVP |
| httpx | Синхронні HTTP-виклики до Nominatim/OSRM з бекенда — ETA MVP (`core/eta.py`, розділ 7.7) | MVP |

## AI / ML

| ТЕХНОЛОГІЯ | РОЛЬ | СТАТУС |
|---|---|---|
| Claude Haiku 4.5 (structured outputs) | Екстракція структурованого фільтра з вільного текстового запиту — NL Search (розділ 7.6) | MVP |
| LLM Gateway (LiteLLM-подібний шар) | Єдина точка виклику моделей різних провайдерів, з фолбеком і лічильником вартості | R&D |
| Claude / GPT-клас моделей (function calling) | Negotiation Agent, структурування документів | R&D |
| Sentence-embedding модель (напр. клас `bge` / `e5`) | Векторизація вантажів і запитів для матчингу й пошуку | R&D |
| Gradient boosting (LightGBM/XGBoost) | Прогноз ринкової ставки, ETA, trust-скоринг — табличні дані, пояснюваність важливіша за глибокі мережі | R&D |
| OCR + layout-модель (напр. клас LayoutLM/Textract) | Розпізнавання BOL/POD/rate confirmation | R&D |
| MLflow / реєстр моделей | Версіонування моделей, відтворюваність | ФАЗА 3 |

## Інфраструктура та спостережуваність

| ТЕХНОЛОГІЯ | РОЛЬ | СТАТУС |
|---|---|---|
| Docker + Docker Compose | Локальний запуск трьох контейнерів (db/backend/frontend) | MVP |
| Kubernetes (керований — EKS/GKE) | Оркестрація в проді, автоскейлінг AI-воркерів окремо від Core | ФАЗА 3 |
| GitHub Actions | CI: лінт, тести, білд образів, деплой | ФАЗА 2 |
| Terraform | Інфраструктура як код (мережа, БД, кластер) | ФАЗА 3 |
| OpenTelemetry + Prometheus/Grafana | Метрики й трейсинг наскрізних запитів, включно з AI-викликами | ФАЗА 3 |
| Sentry | Трекінг помилок фронтенду і бекенду | ФАЗА 2 |

## 05 Доменна модель даних

Поточна схема — дві таблиці, свідомо мінімальні для MVP:

```
-- backend/app/models.py — те, що реально існує сьогодні

users(id, email UNIQUE, hashed_password, company_name,
      role ENUM(shipper|carrier), created_at)

loads(id, title, origin, destination, equipment_type DEFAULT 'Dry Van',
      weight_lbs, price_usd NUMERIC(10,2),
      shipper_id FK→users NULLABLE, carrier_id FK→users NULLABLE,
      shipper_name, carrier_name NULLABLE,
      status ENUM(open|accepted|completed), created_at, accepted_at NULLABLE)
```

> **Вирішено (достроково, ще у Фазі MVP):** `loads` тепер має `shipper_id` / `carrier_id` як FK на `users` — вантаж пов'язаний з автентифікованим користувачем, який його опублікував чи взяв, і саме на це посилання спираються перевірки власності на рівні API (розділ 8). `shipper_name` / `carrier_name` лишаються — це денормалізований кеш для відображення, тепер завжди встановлюється сервером із `company_name` власника, а не клієнтом. Було P0 у розділі 17, тепер закрито.

`accepted_at` (nullable, встановлюється сервером у `accept_load`) додано для ETA MVP (розділ 7.7) — це єдина точка відліку "початку рейсу", яку сьогодні має система, за відсутності телематики чи GPS перевізника. `users.company_name` тепер самостійно редагується власником (`PATCH /api/auth/me`, особистий кабінет на фронтенді, розділ 8) — email і роль лишаються незмінними після реєстрації. Оскільки `loads.shipper_name` / `carrier_name` — денормалізований кеш, а не живий join, перейменування компанії не переписує заднім числом назву на вже опублікованих чи взятих вантажах.

### Цільова схема (Фаза 2–3)

Розширення додає компанії (для trust-скорингу й MC/DOT-номерів), офери та історію переговорів, документи з результатами розпізнавання, знімки прогнозів ставки:

```
CREATE TABLE companies (
  id            BIGSERIAL PRIMARY KEY,
  legal_name    TEXT NOT NULL,
  mc_number     TEXT UNIQUE,      -- Motor Carrier number
  dot_number    TEXT UNIQUE,
  trust_score   NUMERIC(4,1),     -- кеш останнього скору, 0–100
  created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE offers (
  id            BIGSERIAL PRIMARY KEY,
  load_id       BIGINT REFERENCES loads(id),
  carrier_id    BIGINT REFERENCES users(id),
  price_usd     NUMERIC(10,2) NOT NULL,
  status        TEXT NOT NULL DEFAULT 'pending', -- pending|countered|accepted|rejected|e
  source        TEXT NOT NULL DEFAULT 'human',   -- human|ai_agent
  created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE negotiation_messages (
  id              BIGSERIAL PRIMARY KEY,
  offer_id        BIGINT REFERENCES offers(id),
  sender          TEXT NOT NULL,   -- shipper|carrier|ai_agent
  channel         TEXT NOT NULL,   -- chat|email|sms|voice_transcript
  body            TEXT NOT NULL,
  proposed_price  NUMERIC(10,2),
  created_at      TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE trust_scores (
  company_id      BIGINT REFERENCES companies(id),
  score           NUMERIC(4,1) NOT NULL,   -- 0–100
  factors         JSONB NOT NULL,          -- {"payment_history":.., "insurance_valid":..
  model_version   TEXT NOT NULL,
  computed_at     TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (company_id, computed_at)
);

CREATE TABLE market_rate_snapshots (
  lane            TEXT NOT NULL,   -- напр. 'DAL-HOU'
  equipment_type  TEXT NOT NULL,
  predicted_at    TIMESTAMPTZ NOT NULL,
  horizon_days    INT NOT NULL,
  predicted_usd   NUMERIC(10,2) NOT NULL,
  confidence_low  NUMERIC(10,2),
  confidence_high NUMERIC(10,2),
  model_version   TEXT NOT NULL
);
```

```
CREATE TABLE documents (
  id              BIGSERIAL PRIMARY KEY,
  load_id         BIGINT REFERENCES loads(id),
  doc_type        TEXT NOT NULL,   -- bol|pod|rate_confirmation|insurance_coi
  storage_uri     TEXT NOT NULL,   -- s3://...
  extraction      JSONB,           -- структуровані поля з IDP-пайплайну
  confidence      NUMERIC(4,3),
  review_status   TEXT NOT NULL DEFAULT 'auto', -- auto|needs_review|reviewed
  created_at      TIMESTAMPTZ DEFAULT now()
);
```

`loads.embedding vector(768)` додається як колонка pgvector для семантичного пошуку й матчингу (розділ 7.3/7.6) — окрема векторна БД не потрібна, поки обсяг вантажів не вимагає ANN-індексів поза Postgres.

## 06 Сервіси бекенду

| СЕРВІС | ВІДПОВІДАЛЬНІСТЬ | КЛЮЧОВА ТЕХНІКА | СТАТУС |
|---|---|---|---|
| Core Loadboard Service | Користувачі, вантажі, статуси, офери, оплата (майбутнє) | FastAPI + Postgres | MVP |
| Matching Service | Ранжування відкритих вантажів під профіль і історію перевізника | MVP: детерміністична евристика (розділ 7.3); pgvector + gradient-boosted ranking — ціль | MVP |
| Pricing Service | Прогноз ринкової ставки на маршруті на N днів вперед | LightGBM / часові ряди, знімки в `market_rate_snapshots` | R&D |
| Negotiation Service | LLM-агент, що веде першу комунікацію й торгується в заданих межах | LLM function calling + guardrails (розділ 7.1) | R&D |
| Trust Service | Скоринг надійності контрагента, флаги подвійного брокериджу | Табличний класифікатор + зовнішні перевірки (FMCSA, страхові) | R&D |
| Document Intelligence Service | OCR і структурування BOL/POD/rate confirmation | OCR + layout-модель + LLM-екстракція (розділ 7.2) | R&D |
| Search Service | Пошук вантажів природною мовою | LLM parsing (`POST /api/search`, розділ 7.6) → структурований фільтр; векторний пошук ще не реалізовано | MVP |
| ETA Service | Оцінка часу прибуття | MVP: детерміністична евристика на реальній відстані/тривалості через Nominatim+OSRM (розділ 7.7); регресія на телематиці/трафіку/погоді — ціль | MVP |
| Notification Service | Push/email/SMS про зміну статусу вантажу, нові офери | WebSocket + черга задач | ФАЗА 3 |

Усі AI-сервіси — окремі деплойменти, що читають зі спільної шини подій і пишуть результат назад у Postgres через власний вузький API-контракт (не мають прямого доступу до таблиць одне одного).

Це свідомо ближче до модульного моноліту з чіткими межами, ніж до «мікросервісів заради мікросервісів» — команда з 2–5 інженерів не обслужить 9 незалежних репозиторіїв; межі проведені так, щоб їх можна було фізично розрізати пізніше без переписування доменної логіки.

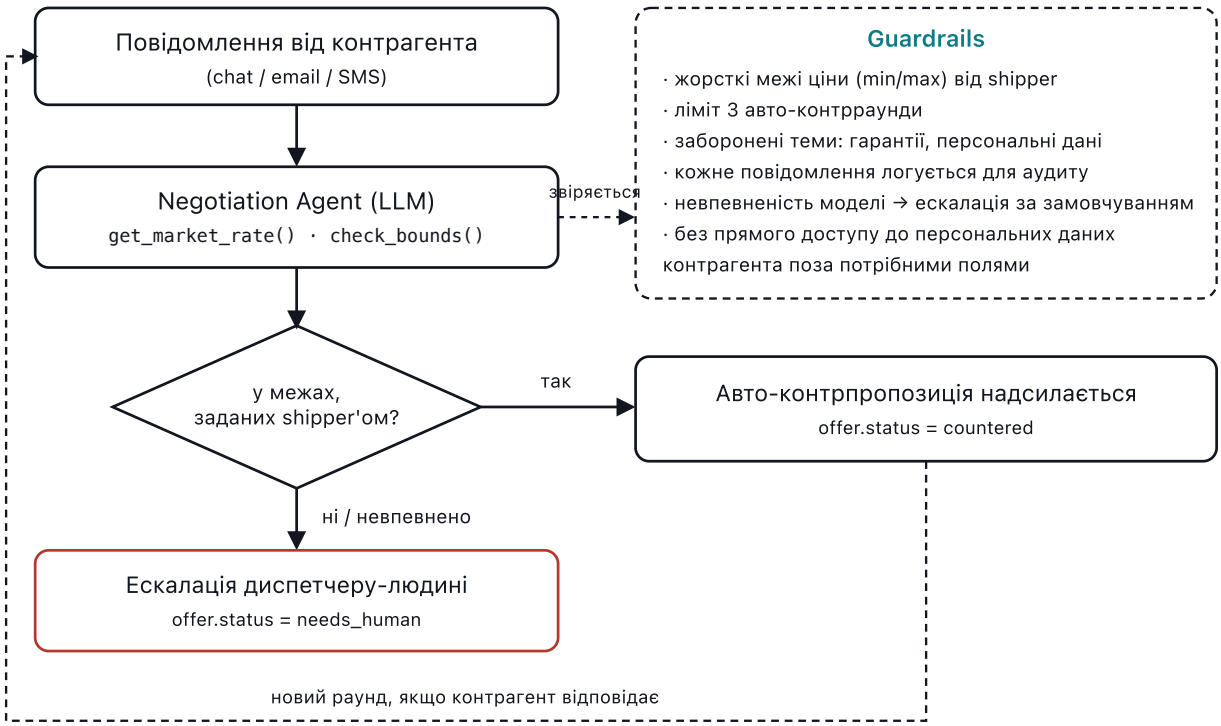
## 07 AI/ML підсистема

Сім продуктових AI-функцій зі спільною інфраструктурою. Спільний принцип для всіх: **модель ніколи не є єдиною точкою відмови** — кожен сервіс має детерміністичний fallback (порожній список рекомендацій, останню відому ставку, ескалацію людині), і жоден AI-виклик не блокує основний транзакційний шлях (розділ 3).

| ФУНКЦІЯ | ПІДХІД | КЛЮЧОВІ ДАНІ | ЛЮДИНА В ЦИКЛІ |
|---|---|---|---|
| Розумний підбір вантажу MVP | MVP: детерміністична евристика за типом кузова й лейнами (embeddings + gradient boosting — ціль) | Маршрут, тип кузова, історія прийнятих вантажів перевізника | Ні — лише сортування, вибір за перевізником |
| Прогноз ринкової ставки | Часові ряди / gradient boosting | Історичні ставки по лейнах, паливо, сезонність, попит/пропозиція | Ні — довірчий інтервал показується явно |
| AI-асистент переговорів | LLM-агент із function calling | Межі ціни від shipper, історія лейну, повідомлення контрагента | Так — ескалація за межами довіри (розділ 7.1) |
| Скоринг надійності | Табличний класифікатор + правила | Історія платежів, FMCSA-дані, страхові документи, флаги шахрайства | Так — низький скор блокує авто-матчинг, потребує ручного review |
| Автоматизація документів | OCR + LLM-екстракція | Скан/фото BOL, POD, rate confirmation | Так — нижче порогу впевненості (розділ 7.2) |
| Пошук природною мовою MVP | LLM parsing → структурований фільтр (векторний пошук — заплановано) | Вільний текстовий запит користувача | Ні — детерміністичний фолбек на пошук за ключовими словами |
| Прогноз ETA MVP | MVP: детерміністична евристика на реальній відстані/тривалості (регресія на телематиці/трафіку/погоді — ціль) | Реальна відстань/тривалість (Nominatim+OSRM), `accepted_at` | Ні |

## 7.1 AI-асистент переговорів

Найризикованіша функція продукту: модель веде реальну комунікацію від імені платформи і може «пообіцяти» ціну. Тому це єдиний AI-сервіс із жорстким контуром безпеки, а не просто LLM-викликом.




Мал. 2 — Агент пропонує ціну лише в межах, які задав shipper заздалегідь; вихід за межі або невпевненість моделі завжди веде до ескалації людині, ніколи не до «здогадки» AI.

**Чому саме так:** LLM за конструкцією не гарантує дотримання числових меж лише інструкцією в промпті — тому перевірка меж винесена в детерміністичний код (`check_bounds()`), а не довірена моделі. Агент може пропонувати формулювання і тон, але не може обійти хардкоджену перевірку суми.

## 7.2 Автоматизація документообігу

Пайплайн розпізнавання BOL/POD/rate confirmation: `upload → OCR/layout → LLM-структурування у JSON → перевірка впевненості → авто-збереження або черга ручної перевірки`. Поле `confidence` з таблиці `documents` (розділ 5) — це мінімум confidence по обов'язкових полях (сума, дата, підпис/номер вантажу); нижче порогу (напр. 0.90) документ іде в чергу ручної перевірки замість автоматичного застосування. Кожне ручне виправлення записується як приклад для донавчання/промпт-тюнінгу — цикл зворотного зв'язку описаний у розділі 13.

## 7.3 Розумний підбір вантажу MVP

**Реалізовано:** `GET /api/loads/matches` (`api/routes/matching.py`) — детерміністична евристика, без LLM і без ML-моделі. Для перевізника з історією (будь-які вантажі, де він `carrier_id`, незалежно від статусу) кожен відкритий вантаж отримує бал: +3 за збіг типу кузова з раніше виконаними вантажами, +2 за збіг штату origin і +2 за збіг штату destination із лейнами перевізника;

результат сортується за балом, потім за часом публікації. Перевізник без історії (новий акаунт) отримує звичайний список найновіших відкритих вантажів — та сама вибірка з БД, що й за `GET /api/loads?status=open`, хоча `/matches` лишається повним масивом без пагінації, на відміну від цього ендпоінту (розділ 8) — це свідомий детермінований випадок ("холодний старт"), а не порожня чи зламана відповідь.

**Чому не embeddings + gradient boosting одразу:** цільова модель нижче потребує реальних поведінкових даних (історія прийнять/відмов у масштабі), яких у демо немає — навчання на кількох сід-вантажах дало б випадковий шум, а не сигнал. Той самий принцип, що й у NL Search (розділ 7.6): спершу детерміністична версія, що дає реальну користь сьогодні, апгрейд до моделі — коли з'являться дані, варті навчання.

## 7.4–7.7 Інші AI-сервіси — принципи проєктування

**`Matching — цільова модель (7.3)`**

Ембедінги вантажу (маршрут, тип кузова, вага, час) + профіль перевізника (домашня база, вподобані лейни, історія прийнять/відмов) → cosine-подібність як базовий сигнал, gradient boosting поверх — для калібрування під фактичну конверсію в прийняття. Замінить евристику вище, коли зберуться реальні дані.

**`Pricing (7.4)`**

Прогноз на лейн-рівні (origin-dest-equipment), горизонт 1–7 днів, з довірчим інтервалом, а не точковим числом — платформа явно показує невпевненість, а не видає її за факт.

**`Trust Scoring (7.5)`**

0–100 з поясненими факторами (JSONB `factors`), не «чорна скринька» — перевізник/shipper бачить, чому скор такий, і може оскаржити. Низький скор не банить автоматично, а знижує пріоритет у матчингу і додає попередження.

**`NL Search (7.6)`** MVP

**Реалізовано:** `POST /api/search` (`core/llm.py`) шле запит у Claude Haiku 4.5 через `messages.parse()` зі структурованим виходом (`SearchFilter`: origin/destination/equipment_type/price_max/weight_min/weight_max), результат застосовується як звичайні SQLAlchemy-фільтри поверх `loads`. Векторний пошук за смисловою близькістю (embeddings) — ще не реалізовано, лишається ціллю Фази 2/3. Сам виклик до LLM вимкнений за замовчуванням прапорцем `NL_SEARCH_ENABLED=false` у `.env` — щоб не витрачати кредити Anthropic, поки хтось свідомо не увімкне; `core/llm.py` перевіряє цей прапорець ще до звернення до ключа. Якщо прапорець вимкнено, ключ `ANTHROPIC_API_KEY` відсутній, виклик до Anthropic падає або на акаунті скінчились кредити — фолбек не «форма з фільтрами» і не нефільтрований список, а простий пошук за ключовими словами (`_keyword_filter`): кожне слово запиту через `ILIKE` має збігтися хоч в одній із колонок title/origin/destination/equipment_type; ендпоінт ніколи не повертає помилку через відмову LLM і завжди хоч якось фільтрує.

**`ETA (7.7)`** MVP

**Реалізовано:** `GET /api/loads/{id}/eta` (`core/eta.py`) — детерміністична евристика, без ML-моделі. Геокодує origin/destination через Nominatim, отримує реальну відстань і тривалість руху через OSRM (ті самі провайдери, що й карта на фронтенді, тут викликаються з бекенда), і розширює звичайну (легковою) тривалість OSRM у діапазон `[drive_hours_min, drive_hours_max]` (×1.3) — наближення для типових обов'язкових зупинок на відпочинок під час далекого рейсу. Коли перевізник бере вантаж (`loads.accepted_at`, встановлюється сервером у `accept_load`), ендпоінт додає цей діапазон до `accepted_at` і повертає вікно прибуття (`eta_earliest`/`eta_latest`); до

прийняття — лише оцінку часу в дорозі, бо немає точки відліку відправлення. На сторінці деталей вантажу відображається як панель «Estimated arrival» після прийняття вантажу.
**Чому не справжня модель одразу:** цільова модель потребує GPS-телематики перевізника, живого трафіку й погоди — нічого з цього в демо немає. Той самий принцип, що й у Matching (7.3) та NL Search (7.6): спершу детерміністична версія, що дає реальну користь сьогодні.

**Спільне для всіх**

Кожна модель має версію (`model_version`), логується вхід/вихід для офлайн-оцінки, і має задокументований fallback на детермінізм при відмові.

## 7.8 Спільна AI-інфраструктура

- **LLM Gateway** — єдина точка виклику моделей: абстрагує провайдера (можливість перемкнутись між моделями без зміни коду сервісів), рахує вартість токенів на запит, застосовує тайм-аути й fallback на меншу/дешевшу модель при відмові основного провайдера.
- **Редагування PII перед відправкою назовні.** Персональні дані водія/контактні дані контрагента маскуються перед тим, як текст іде до стороннього LLM-провайдера, і відновлюються після відповіді — вимога для документообігу з ПІБ водіїв і номерами телефонів.
- **Реєстр промптів і eval-набори.** Кожен промпт версіонується; зміна промпту прогонюється проти зафіксованого набору реальних кейсів (regression suite для AI, а не лише для коду) перед викаткою.
- **Canary rollout моделей.** Нова версія моделі/промпту спершу отримує невелику частку трафіку, порівнюється за метриками якості й вартості, і лише потім масштабується на 100%.
- **Бюджет вартості й circuit breaker.** Ліміт витрат на AI-виклики за період; при перевищенні — деградація до дешевших/детерміністичних шляхів, а не мовчазне зростання рахунку. `MVP` Для NL Search уже є найпростіша форма: денний ліміт викликів Anthropic (`NL_SEARCH_DAILY_LIMIT`, дефолт 500 на UTC-добу, `backend/app/core/llm.py`) — після вичерпання пошук тихо деградує до keyword-фільтра. Рахується за кількістю викликів, не за $/токенами, і в пам'яті процесу (розділ 17, пункт 17); повноцінний бюджет у $ і circuit breaker на помилки провайдера — ціль.
- **Feature Store.** Спільні ознаки (історія лейну, поведінка перевізника) обчислюються один раз і перевикористовуються Matching, Pricing і Trust сервісами замість дублювання логіки.

---

# 08 API

## Конвенції

- Базовий шлях `/api`, JSON тіла запитів/відповідей, помилки — `{"detail": "..."}` (стандартний формат FastAPI/Pydantic).
- Автентифікація — заголовок `Authorization: Bearer <JWT>`.

- Rate limiting: `POST /api/auth/register`, `/login` і `POST /api/search` мають per-IP ліміт; понад нього — `429` із заголовком `Retry-After` (секунди) і тим самим форматом `{"detail": "..."}` (розділ 10).
- Публічна специфікація автогенерується FastAPI: `/docs` (Swagger UI) та `/openapi.json`.
- Заплановано на Фазу 2: версіонування шляху (`/api/v1/...`), ідемпотентні ключі для `POST /offers`.
- `GET /api/loads` уже має пагінацію — MVP-версія обрала просту сторінкову (offset) пагінацію через `?page=` / `?page_size=` (розділ нижче) замість курсорної (`?cursor=`), яку цей документ раніше називав ціллю Фази 2. Offset-пагінація простіша в реалізації й дає звичний UI "сторінка X з Y", але деградує на дуже великих і часто змінюваних наборах даних (зсув сторінки "пливе" при вставках/видаленнях, а `OFFSET` на глибоких сторінках сканує й відкидає рядки). Курсорна пагінація лишається ціллю Фази 2 — саме на той момент, коли обсяг вантажів реально виправдає складнішу реалізацію.

## Реалізовані ендпоінти (MVP)

| МЕТОД | ШЛЯХ | ОПИС | AUTH |
|---|---|---|---|
| `GET` | `/api/health` | Перевірка живості сервісу | — |
| `POST` | `/api/auth/register` | Реєстрація (email, пароль, компанія, роль) — видає пару access+refresh | — |
| `POST` | `/api/auth/login` | Вхід — видає пару access-JWT (15 хв) + refresh-токен | — |
| `GET` | `/api/auth/me` | Поточний користувач за токеном | Bearer |
| `PATCH` | `/api/auth/me` | Редагування профілю — лише `company_name` (email/роль незмінні); не перейменовує заднім числом `shipper_name` / `carrier_name` на вже опублікованих/взятих вантажах — це денормалізований кеш, розділ 5 | Bearer |
| `POST` | `/api/auth/refresh` | Обміняти дійсний refresh-токен на нову пару (ротація, розділ 9) | — |
| `POST` | `/api/auth/logout` | Відкликати refresh-токен | — |
| `GET` | `/api/loads` | Список вантажів, опційний фільтр `?status=`, пагінація `?page=` / `?page_size=` (за замовчуванням 1/20, максимум 100) — сторінка за межами останньої повертає порожній `items` (200, не 404) | — |
| `GET` | `/api/loads/mine` | Особистий кабінет — усі вантажі, де користувач фігурує з будь-якого боку (опублікував як shipper або взяв як carrier), без обмеження за роллю; без пагінації (особистий список, не основна таблиця перегляду) | Bearer |
| `GET` | `/api/loads/{id}` | Деталі одного вантажу — сторінка деталей на фронтенді | — |
| `POST` | `/api/loads` | Публікація вантажу — тільки роль `shipper`, `shipper_id` береться з токена | Bearer |

| МЕТОД | ШЛЯХ | ОПИС | AUTH |
|---|---|---|---|
| `POST` | `/api/loads/{id}/accept` | Взяти вантаж напряму — тільки роль `carrier`, `carrier_id` береться з токена | Bearer |
| `POST` | `/api/loads/{id}/complete` | Позначити завершеним — лише shipper вантажу або carrier, що його взяв | Bearer |
| `POST` | `/api/search` | Пошук вантажів природною мовою — LLM-парсинг у фільтр (розділ 7.6), фолбек на пошук за ключовими словами | — |
| `GET` | `/api/loads/matches` | Розумний підбір вантажу — детерміністична евристика під історію перевізника (розділ 7.3) | Bearer, роль `carrier` |
| `GET` | `/api/loads/{id}/eta` | Прогноз часу прибуття — детерміністична евристика на реальній відстані/тривалості (розділ 7.7) | — |

### Заплановані ендпоінти (Фаза 2–3)

| МЕТОД | ШЛЯХ | ОПИС |
|---|---|---|
| `GET` | `/api/rates/forecast` | Прогноз ставки за лейном/типом кузова/датою |
| `POST` | `/api/loads/{id}/offers` | Створити офер (людиною або AI-агентом) |
| `POST` | `/api/offers/{id}/counter` | Контрпропозиція |
| `GET` | `/api/companies/{id}/trust-score` | Скор надійності з поясненими факторами |
| `POST` | `/api/loads/{id}/documents` | Завантажити BOL/POD/rate confirmation |
| `WS` | `/ws/notifications` | Realtime-сповіщення: нові офери, зміни статусу |

## 09 Автентифікація, авторизація й ролі

Сьогодні: email+пароль → `bcrypt`-хеш → короткоживучий access-JWT (HS256, `sub`=email, 15 хв за замовчуванням) з `python-jose` + окремий refresh-токен (опаковий випадковий рядок, зберігається в БД лише як SHA-256-хеш, таблиця `refresh_tokens`). Ротація на кожному використанні (`POST /api/auth/refresh`) — старий refresh-токен помирає в момент видачі нового, тож повторне використання вкраденого токена після того, як легітимний клієнт уже зробив refresh, — відхилене

повторне використання, а не тиха друга сесія. `POST /api/auth/logout` відкликає токен явно. Сам access-JWT відкликати не можна (він статичний за конструкцією), але коротке 15-хвилинне вікно замість 7 днів суттєво звужує вікно ризику. Фронтенд оновлює access-токен у фоні непомітно для користувача (`AuthContext.tsx`). Обидва токени зберігаються в `localStorage` — це усвідомлений компроміс, не цільовий дизайн (див. нижче).

| РОЛЬ | МОЖЕ СЬОГОДНІ | ЦІЛЬ (ФАЗА 2+) |
|---|---|---|
| shipper | Публікувати вантаж | + бачити оферти, вести переговори, бачити trust-скор перевізника |
| carrier | Брати відкритий вантаж | + AI-рекомендації, робити контрпропозиції, завантажувати POD |
| dispatcher (enterprise) | — | Керує кількома вантажівками/водіями від імені carrier-компанії |
| admin | — | Модерація, перегляд журналу аудиту, ручний review документів/скорингу |

Цільові покращення: refresh-токен через httpOnly cookie замість `localStorage` (усуває XSS-читання, потребує CSRF-захисту навзаєм), відкликання через deny-list у Redis замість таблиці Postgres (масштаб), MFA для enterprise-акаунтів, SSO (OAuth2/OIDC) для великих shipper-клієнтів, окремі API-ключі з обмеженими scope для інтеграцій (TMS-конектори).

## 10 Безпека

| ОБЛАСТЬ | ПОТОЧНИЙ СТАН | ЦІЛЬ |
|---|---|---|
| Секрети | `JWT_SECRET` зі змінної середовища, дефолт `"dev-secret-change-me"` у коді | Секрет-менеджер (Vault/AWS Secrets Manager), без дефолтів у коді |
| CORS | Явний allowlist через `CORS_ORIGINS` | Домени продакшн-фронтенду замінюють дефолтний dev-origin |
| Rate limiting | In-memory sliding window по IP клієнта (`backend/app/core/rate_limit.py`): `/api/auth/register` + `/login` — спільний бюджет 10/хв (`AUTH_RATE_LIMIT_PER_MINUTE`), `POST /api/search` — 20/хв (`SEARCH_RATE_LIMIT_PER_MINUTE`); понад ліміт — HTTP 429 з `Retry-After`. `/refresh` свідомо без ліміту (256-бітний токен, нічого підбирати). Стан на процес, не спільний (розділ 17, пункт 17) | На рівні API Gateway або спільне сховище (Redis), окремо жорсткіше на `/auth/*` |
| Авторизація на рівні ресурсу | Є на трьох mutating-ендпоінтах вантажів (розділ 17, закрито) | Те саме покриття по мірі появи нових ресурсів (offers, documents) |
| PII та документи | Н/З — документів ще немає | Шифрування в спокої (S3 SSE), маскування PII перед LLM-викликами (розділ 7.8) |
| Аудит | Відсутній | `audit_log`: хто/що/коли, окремо — усі AI-повідомлення від імені платформи (розділ 7.1) |
| Залежності | Не скануються | Dependabot/`pip-audit` у CI, блокування критичних CVE |
| Пентест | Не проводився (демо-стадія) | Перед публічним запуском і раз на рік/після значних змін |

## 11 Інфраструктура та DevOps

Сьогодні: `docker-compose.yml` піднімає три контейнери — `db` (Postgres 16 з healthcheck), `backend` (FastAPI, ретраї підключення до БД при старті), `frontend` (Vite dev-сервер). Один хост, без окремих середовищ.

| СЕРЕДОВИЩЕ | ПРИЗНАЧЕННЯ | СТАТУС |
|---|---|---|
| local | Розробка через `docker compose up` | MVP |
| staging | Перевірка перед релізом, синтетичні дані | ФАЗА 2 |
| production | Kubernetes, окремі node pool для Core і AI-воркерів (AI-воркери — з доступом до GPU/вищим CPU-лімітом і власним автоскейлінгом за чергою подій) | ФАЗА 3 |

**CI/CD (Фаза 2):** GitHub Actions — лінт (ruff/eslint) → тести → білд Docker-образів → пуш у реєстр → деплой у staging автоматично, у production через ручне затвердження. **Резервне копіювання:** щоденний снапшот Postgres + point-in-time recovery до 7 днів; документи в об'єктному сховищі — versioning увімкнено для захисту від випадкового перезапису.

## 12 Продуктивність і масштабованість

- **Кешування.** Redis для «гарячого» списку відкритих вантажів і результатів прогнозу ставки (TTL кілька хвилин — прогноз не повинен перераховуватись на кожен рендер списку).
- **Асинхронні задачі.** Важкі AI-виклики (переговори, OCR) йдуть через чергу (Celery/RQ поверх Redis, пізніше — Kafka), не в request-response циклі API.
- **Індекси БД.** `loads(status, created_at)` складений індекс для основного списку; `loads(origin, destination, equipment_type)` для матчингу; pgvector HNSW-індекс на `embedding`.
- **Read-репліки.** Читання списку вантажів і аналітики — з репліки, запис — лише на primary, коли навантаження це виправдає.
- **CDN.** Статика фронтенду й завантажені документи (через підписані URL) — за CDN, не проксуються бекендом.

| МЕТРИКА | ЦІЛЬОВИЙ БЮДЖЕТ |
|---|---|
| Список вантажів, p95 | `< 200 мс` |
| Публікація вантажу, p95 | `< 300 мс` |
| AI-матчинг (асинхронний), p95 | `< 3 с` |
| Перший хід Negotiation Agent, p95 | `< 8 с` |
| Доступність Core API | `99.9%` |

## 13 Спостережуваність

Сьогодні — лише stdout-логи `uvicorn`, нічого структурованого. Цільовий стек:

- **Логи** — структуровані (JSON), з `request_id`, що проходить через Gateway → Core → подію → AI-сервіс, для наскрізного трасування одного запиту користувача.
- **Метрики** — Prometheus: латентність по ендпоінтах, довжина черги подій, вартість AI-викликів у доларах на добу.
- **Трейсинг** — OpenTelemetry, окремо позначені AI-спани (модель, кількість токенів, латентність провайдера).
- **AI-специфічна спостережуваність:** eval-пайплайн на semple реальних кейсів після кожної зміни промпту/моделі; дрейф якості (частка ескалацій Negotiation Agent людині — раптове зростання сигналізує про регресію); черга людського review для документів і скорингу як явний UI, а не побічний ефект.
- **Цикл зворотного зв'язку.** Кожне ручне виправлення (диспетчер поправив офер AI, людина виправила поле документа) записується як приклад для наступного eval-набору й потенційного донавчання/промпт-тюнінгу.

## 14 Зовнішні інтеграції

| КАТЕГОРІЯ | НАВІЩО | ВИКОРИСТОВУЄТЬСЯ В |
|---|---|---|
| Картографія / трафік | Відстань, час у дорозі, затори | ETA, Matching (порожній пробіг) |
| Погода | Коригування ETA, ризик затримки | ETA |
| ELD / телематика перевізника | Реальна GPS-позиція, HOS-ліміти водія | ETA, Trust |
| FMCSA / кредитні бюро | MC/DOT-верифікація, страхове покриття, історія претензій | Trust Scoring, онбординг компанії |
| e-Signature | Підписання rate confirmation без обміну PDF поштою | Document Intelligence, офери |
| Платежі / факторинг | Швидка оплата перевізнику після POD (майбутня монетизація) | Завершення вантажу |
| SMS / email провайдер | Сповіщення, канал для Negotiation Agent | Notification, Negotiation |
| DAT/Truckstop (публічні API, де доступно) | Холодний старт ринкових даних для Pricing до накопичення власної історії | Pricing Service |

## 15 Тестування та якість

| РІВЕНЬ | ІНСТРУМЕНТ | ЩО ПОКРИВАЄ | СТАТУС |
|---|---|---|---|
| Unit (бекенд) | pytest | Схеми, бізнес-правила (переходи статусу вантажу), геокодування/маршрутизація (мокані `httpx`-виклики) — 99% покриття рядків `app/`, 69 тестів | MVP |
| Unit (фронтенд) | Vitest + Testing Library | Перший зріз — `AuthContext` (bootstrap через refresh, logout), `EtaWindow`, `AuthPanel`, `haversineMiles`, `Combobox` (пошук/дебаунс/клавіатура), `LoadsPage` (пошук state→city для постингу вантажу), `api.ts` (усі виклики бекенду, включно з обробкою помилок), `App.tsx` (шапка/навігація, статус авторизації, модалка логіну, маршрутизація), `HomePage` (статистика вантажів, CTA-кнопки залежно від авторизації), `LoadDetailPage` (стани завантаження/помилки, поля деталей, кожна гілка сценарію прийняття вантажу), `DocsPage` (обидві картки документів, посилання на HTML/PDF кожної картки, безпечні атрибути `target="_blank"` / `rel="noopener noreferrer"`), `RouteMap` (Leaflet-мапа — `react-leaflet` мокається на рівні модуля, щоб jsdom не рендерив справжню мапу; машина станів завантаження/недоступності/готовності, форматування відстані/тривалості, розгалуження суцільна-vs-пунктирна лінія, захист від застарілої відповіді при зміні пропсів, 100% покриття рядків/гілок), і `ProfilePage` (особистий кабінет — профіль, редагування назви компанії, список власних вантажів зі статистикою за статусами): 13 файлів, 106 тестів — кожна сторінка/компонент, названі в цьому пункті технічного боргу, тепер мають хоча б часткове покриття | MVP |
| Інтеграційні | pytest + реальний Postgres у Docker | Ендпоінти API наскрізь через БД — той самий набір тестів, що й Unit вище; проєкт поки не розділяє ці два рівні окремими сьютами | MVP |

| РІВЕНЬ | ІНСТРУМЕНТ | ЩО ПОКРИВАЄ | СТАТУС |
|---|---|---|---|
| E2E | Playwright | Реєстрація → публікація → взяття вантажу | ФАЗА 3 |
| AI-eval | Власний харнес поверх зафіксованого набору кейсів | Якість Negotiation/Matching/Trust — регресія перед кожним релізом промпту чи моделі | R&D |
| Навантажувальні | k6/Locust | Латентність під пік-навантаженням, черга подій під бекпресурою | ФАЗА 3 |

Сьогодні: `backend/tests` — 11 файлів, 69 тестів, 99% покриття рядків `app/` (кожен модуль `api/routes/*` і `core/eta.py` — 100%): health-check; auth/роль/власність на трьох мутуючих ендпоінтах вантажів разом із повним переходом статусів (`open→accepted→completed`, включно з тим, що вантаж не можна завершити, не взявши його спершу — цієї перевірки бракувало до цього заходу, закрито в тій самій зміні, розділ 17); `POST /api/search` (виклик Anthropic мокається на місці імпорту, включно з власною гілкою відмови — тести детерміністичні, без мережі й без `ANTHROPIC_API_KEY`); `GET /api/loads/matches` (ранжування, роль, холодний старт); `GET /api/loads/{id}` (деталі, 404, регресійний тест на те, що `{load_id:int}` не "проковтує" `/matches`); `GET /api/loads/{id}/eta` на двох рівнях — сам ендпоінт (мокана `estimate_transit`) і евристика під ним (`core/eta.py`, мокан `httpx.get` — реальна логіка геокодування/маршрутизації/кешу/фолбеків виконується насправді); повний цикл refresh-токена (видача, ротація, відхилення повторного використання, відкликання); `PATCH /api/auth/me` (зміна назви компанії, 422 на порожнє значення, і регресійний тест на те, що перейменування не переписує заднім числом `shipper_name` на вже опублікованих вантажах); і `GET /api/loads/mine` (потребує токен, shipper бачить лише свої опубліковані, carrier — лише свої взяті, сортування від найновіших). Фронтенд: Vitest + Testing Library, перший зріз — 13 файлів, 106 тестів (див. таблицю вище); кожна сторінка/компонент, названі як прогалина в розділі 17, тепер мають хоча б часткове покриття, хоча покриття всередині файлів усе ще не повне.

## 16 Технічна дорожня карта

| ФАЗА | ТЕХНІЧНА ЦІЛЬ | AI-ФІЧІ, ЩО ВМИКАЮТЬСЯ |
|---|---|---|
| **MVP** ГОТОВО | Транзакційний каркас: auth, вантажі, статуси | Пошук природною мовою (базова версія — структурований фільтр, без векторного пошуку) і розумний підбір вантажу (базова версія — детерміністична евристика, без ML) ГОТОВО |
| **Фаза 2** 3–4 MIC | Авторизація на рівні ресурсу, тести, CI, черга задач, об'єктне сховище, pgvector | Embeddings + gradient boosting для Matching, векторний пошук поверх NL Search, rate limiting/cost cap для LLM-викликів |
| **Фаза 3** 5–9 MIC | Kubernetes, шина подій, спостережуваність, мобільний застосунок | Прогноз ставки, скоринг надійності, автоматизація документів, ETA |
| **Фаза 4** R&D | Multi-region, SSO для enterprise, платіжний рейл | AI-асистент переговорів (найризикованіша фіча — останньою, після довіри користувачів до простіших AI-функцій) |

Порядок навмисний: Negotiation Agent — найцінніша, але й найризикованіша фіча (модель говорить від імені платформи з реальними грошима на кону). Вона йде останньою, коли інфраструктура guardrails, аудиту й ескалації (розділ 7.1, 13) вже перевірена на менш ризикованих AI-функціях.

## 17 Технічний борг і відомі обмеження

| # | ПРОБЛЕМА | ВПЛИВ | ПРІОРИТЕТ |
|---|---|---|---|
| 1 | ~~`POST /api/loads`~~, ~~`/accept`~~, ~~`/complete`~~ ~~не перевіряли автентифікацію чи роль~~ — закрито: усі три вимагають Bearer-токен і перевіряють роль (розділ 8) | Було: будь-хто без токена міг публікувати/ забирати/закривати вантажі | ЗАКРИТО |
| 2 | ~~`loads.shipper_name`~~ / ~~`carrier_name`~~ ~~— вільний текст, не ГК на~~ ~~`users`~~ — закрито: додано `shipper_id` / `carrier_id` (розділ 5) | Було: неможливо перевірити власність вантажу; дані легко підробити | ЗАКРИТО |
| 3 | ~~JWT без refresh і без відкликання~~ — закрито: короткоживучий (15 хв) access-JWT + окремий revocable refresh-токен з ротацією (розділ 9, `backend/app/core/security.py`). Дефолтний `JWT_SECRET` у коді лишається — свідомий виняток лише для демо (security.md), не забути замінити перед будь-яким спільним/staging-оточенням (див. README) | Було: вкрадений токен дійсний 7 днів без способу його відкликати | ЗАКРИТО |
| 4 | ~~CORS~~ ~~`allow_origins=["*"]`~~ — закрито: явний allowlist через змінну середовища `CORS_ORIGINS` (`backend/app/core/config.py`, `.env.example`), уже застосовувалося на момент попереднього запису в цій таблиці — рядок просто не оновили тоді (розділ 10) | Було: будь-який origin міг робити крос-доменні запити до API | ЗАКРИТО |
| 5 | ~~Схема БД створюється через~~ ~~`Base.metadata.create_all`~~ ~~, без міграцій~~ — закрито: Alembic (`backend/migrations/`), застосовується перед стартом застосунку в усіх середовищах (розділ 4) | Було: немає керованого шляху еволюції схеми в продакшн-даних | ЗАКРИТО |
| 6 | ~~Тести бекенду тонкі, фронтенд-тестів не існує взагалі~~ — бекенд закрито: 99% покриття рядків `app/`, 69 тестів (у процесі знайдено й закрито реальний баг: `complete_load` дозволяв завершити вантаж, який ніколи не брали). Для фронтенду закрито інфраструктуру, не покриття: Vitest + Testing Library підключені в CI, перший зріз — 13 файлів, 106 тестів | Було: більшість бізнес-логіки без регресійного захисту; покриття всередині файлів усе ще не повне, і будь-яка нова | P1 |

| # | ПРОБЛЕМА | ВПЛИВ | ПРІОРИТЕТ |
|---|---|---|---|
| | (`AuthContext`, `EtaWindow`, `AuthPanel`, `haversineMiles`, `Combobox`, `LoadsPage`, `api.ts`, `App.tsx`, `HomePage`, `LoadDetailPage`, `DocsPage`, `RouteMap`, `ProfilePage`), розділ 15 — кожна сторінка/компонент, названі тут спочатку, тепер мають хоча б часткове покриття | сторінка знову починає з нуля | |
| 7 | ~~Немає rate limiting на `/api/auth/*`~~ — закрито: `/register` і `/login` ділять один per-IP бюджет (10/хв за замовчуванням, `AUTH_RATE_LIMIT_PER_MINUTE`), понад нього — 429 з `Retry-After` (розділ 10, `backend/app/core/rate_limit.py`). `/refresh` свідомо поза бюджетом: 256-бітний токен не підібрати, а 429 там фронтенд прочитав би як мертву сесію й розлогінив користувача. Обмеження реалізації (стан у пам'яті процесу) — окремий пункт 17 | Було: вразливість до brute-force підбору пароля | ЗАКРИТО |
| 8 | 4 із 7 AI-сервісів — поки що специфікація, не код (реалізовано NL Search, Matching і ETA MVP, розділи 7.3/7.6/7.7 — усі три детерміністичні, без ML/embeddings) | Основна цінність продукту для інвестора здебільшого ще не побудована | ВІДКРИТО |
| 9 | ~~`POST /api/search` без rate limiting і без per-request/per-period cost cap на виклики Anthropic~~ — закрито: per-IP ліміт 20/хв (`SEARCH_RATE_LIMIT_PER_MINUTE`) плюс глобальний денний ліміт викликів Anthropic (`NL_SEARCH_DAILY_LIMIT`, 500/UTC-добу), після якого пошук деградує до keyword-фільтра, а не до помилки (розділ 7.8). Per-request стелю вже давав `max_tokens`. Повноцінний $-бюджет і circuit breaker розділу 7.8 лишаються ціллю; обмеження «в пам'яті процесу» — пункт 17 | Було: будь-хто без автентифікації міг ініціювати необмежену кількість платних LLM-викликів | ЗАКРИТО |
| 10 | `frontend/src/geocode.ts` викликає Nominatim напряму з браузера, без бекенд-проксі й без rate limiting поза кешем в пам'яті на сесію — політика Nominatim обмежує безкоштовне використання ~1 запитом/сек | Реальний продакшн-трафік ризикує блокуванням IP з боку Nominatim | P2 |

| # | ПРОБЛЕМА | ВПЛИВ | ПРІОРИТЕТ |
|---|---|---|---|
| 11 | `frontend/src/routing.ts` викликає публічний demo-сервер OSRM напряму з браузера — той самий тип обмеження, що й пункт 10: без бекенд-проксі, без rate limiting поза кешем у сесії; сервер офіційно не призначений для продакшн-навантаження | Немає SLA/гарантії доступності для реального трафіку | P2 |
| 12 | `backend/app/core/eta.py` також викликає Nominatim і публічний demo-сервер OSRM напряму (з бекенда, для ETA MVP, розділ 7.7) — той самий тип обмеження, що й пункти 10–11, але тепер і серверний трафік; кеш у пам'яті процесу (не за сесію) частково пом'якшує навантаження на ці сервіси, але не є заміною реальному rate limiting чи платному провайдеру | Той самий ризик блокування/ відсутності SLA, тепер із серверного, а не лише клієнтського боку | P2 |
| 13 | Refresh-токен (розділ 9) зберігається в `localStorage`, як і access-токен до нього — читається будь-яким JS на сторінці. Ротація (пункт 3 вище) обмежує вкрадений токен одним використанням, але не запобігає самій крадіжці; цільовий дизайн — httpOnly cookie, яку клієнтський JS взагалі не бачить, разом із CSRF-захистом | XSS будь-де на сайті може вкрасти сесію на весь термін дії refresh-токена (30 днів за замовчуванням), а не лише на 15 хв access-токена | P2 |
| 14 | ~~`frontend`'s `vite@^5.4.0` транзитивно тягнув вразливий `esbuild` (GHSA-67mh-4wv8-2f99)~~ — закрито: `vite` оновлено `^5.4.0` → `^8.2.1` (разом із `@vitejs/plugin-react` → `^6.0.5` і `vitest` / `@vitest/coverage-v8` → `^4.1.10` — усі чотири мали рухатись разом, розділ 15). `npm audit` тепер показує 0 вразливостей; Vite 8 більше не залежить від `esbuild` узагалі, не просто пропатчену версію. Перевірено: повний тест-сьют, лінт, білд і Docker-образ (`node:20-alpine` → Node 20.20.2, вище мінімуму Vite 8 `^20.19.0`) — усе пройшло без інших змін коду | Було: дев-сервер приймав крос-origin запити й повертав відповідь | ЗАКРИТО |
| 15 | `frontend/src/placeSearch.ts` викликає Photon (`photon.komoot.io`, keyless публічний геокодер Komoot) напряму з браузера — живий, дебаунсений пошук міст/сіл/селищ для пікера міста у формі постингу вантажу. Той самий тип обмеження, що й пункти geocode.ts/routing.ts/core-eta.py вище: без ключа, без бекенд-проксі, без rate limiting поза кешем | Четвертий сторонній картографічний сервіс без продакшн-гарантій, від якого залежить застосунок (Nominatim ×2 точки | P2 |

| # | ПРОБЛЕМА | ВПЛИВ | ПРІОРИТЕТ |
|---|---|---|---|
| | у пам'яті на сесію, і це явно публічний demo-інстанс без гарантій доступності. Свідомо інший провайдер, ніж Nominatim (який використовується для точного геокодування в інших місцях): Nominatim не робить справжній prefix/typeahead-пошук (перевірено напряму — запит "Chicag" під час набору "Chicago" не повертає нічого), Photon побудований саме для автодоповнення | виклику, OSRM ×2, тепер Photon) | |
| 16 | `GET /api/loads` отримав пагінацію (`?page=` / `?page_size=`, розділ 8) — `GET /api/loads/matches` (matching.py) і `POST /api/search` (search.py) свідомо лишились поза цим змінами й досі повертають повний, непагінований масив: обидва звертаються до БД напряму, не через ендпоінт списку. Прийнятно зараз (демо-масштаб), але при зростанні кількості вантажів обидва відгуки ростимуть без обмеження — той самий клас проблеми, який пагінація щойно закрила для `GET /api/loads` | Розмір відповіді й час запиту цих двох ендпоінтів необмежено ростуть разом з кількістю вантажів | P2 |
| 17 | Rate limiting (пункти 7, 9) і денний ліміт NL Search живуть у пам'яті процесу: при N uvicorn-воркерах/ репліках фактичний ліміт — N × налаштований, рестарт обнуляє лічильники. Ключ — TCP-адреса клієнта; `X-Forwarded-For` свідомо ігнорується (його контролює клієнт), тож за reverse-proxy або в Docker Desktop (port-forwarding) усі клієнти можуть ділити одну адресу й один бюджет. Ціль — API Gateway або Redis за тим самим інтерфейсом, плюс trusted-proxy конфіг | Ліміти слабші, ніж налаштовано, при горизонтальному масштабуванні; за проксі — спільний бюджет на всіх | P2 |

Пункти 1–2 закрито до початку роботи над AI, як і планувалося: без FK-зв’язку вантажу з власником неможливо було б коректно побудувати ані trust scoring, ані персоналізований matching, ані журнал переговорів — уся AI-підсистема з розділу 7 спирається саме на цей зв’язок.